

511 Exploration Data Analysis

2026-05-03

EDA Data Cleaning

```
# =====  
# Source: https://trails-wa-rco.hub.arcgis.com  
  
# df_q1 | COMPARATIVE | surface type vs. segment length & width by use  
# df_q2 | CORRELATION | trail width vs. number of permitted use types  
# df_q3 | GEOGRAPHIC | motorized vs. non-motorized trail miles by county  
# df_q4 | TIME/TREND | construction volume over time by trail type & agency  
#  
# Variable inventory per sub-dataset:  
#  
# df_q1 objectid, trail_name, surface_cat, use_cat,  
#       segment_length_miles, trail_width  
#  
# df_q2 objectid, trail_name, trail_width, num_uses,  
#       motor_type, [13 binary use flags]  
#  
# df_q3 objectid, trail_name, county, region,  
#       is_motorized, segment_length_miles  
#  
# df_q4 objectid, trail_name, year_constructed, decade, era,  
#       trail_type_cat, agency_clean, segment_length_miles  
#  
# =====
```

```
# 0. LIBRARIES  
library(tidyverse)
```

```
## Warning: package 'lubridate' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
## v dplyr      1.1.4      v readr      2.1.5  
## v forcats    1.0.0      v stringr    1.5.1  
## v ggplot2    3.5.1      v tibble     3.2.1  
## v lubridate  1.9.5      v tidyr      1.3.1
```

```
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()   masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(janitor)
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##   chisq.test, fisher.test
```

```
library(skimr)
```

```
## Warning: package 'skimr' was built under R version 4.4.3
```

```
# 1. LOAD & INSPECT
```

```
trails_raw <- read_csv("trails.csv", show_col_types = FALSE)
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
trails <- trails_raw |> clean_names() # → snake_case column names
```

```
cat("Raw:", nrow(trails), "rows x", ncol(trails), "cols\n")
```

```
## Raw: 22338 rows x 51 cols
```

```
glimpse(trails)
```

```
## Rows: 22,338
## Columns: 51
## $ objectid      <dbl> 32305, 32511, 32512, 32513, 32514, 32515, 3~
## $ trail_name     <chr> "Rufus", NA, "Bill Chipman Palouse Trail", ~
## $ trail_alternate_name <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ trail_number   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ segment_id     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ segment_length_miles <dbl> 0.02066238, 1.83708130, 13.86535544, 2.0383~
## $ trail_system_type <chr> "State", "State", "State", "State", "State"~
## $ trail_system_name <chr> NA, NA, NA, NA, NA, "Palouse to Cascades", ~
## $ national_trail_designation <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ trail_surface   <chr> NA, "Natural", "Natural", "Asphalt", "Natur~
## $ trail_condition <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ trail_type      <chr> NA, "Standard Terra Trail", "Standard Terra~
```

```
## $ trail_class      <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ primary_use     <chr> NA, "Hiking", "Hiking", "Hiking", "Hiking", ~
## $ trail_width     <dbl> NA, NA, NA, 10, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ trail_usage     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ seasonality     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ season_notes    <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ directionality  <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ speed_limit_mph <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ accessibility_status <chr> "Not Evaluated", NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ special_management_area <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "Other", "O~
## $ interpretive_trail <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ rail_trail      <chr> NA, NA, NA, NA, NA, NA, "yes", NA, NA, NA, NA, ~
## $ hiking_walking  <chr> NA, "yes", "yes", "yes", "yes", "yes", "yes", ~
## $ bicycle         <chr> NA, "Yes", "Yes", "Yes", "Yes", "yes", "yes", ~
## $ e_bike          <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ x4wd            <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "No", "Unkn~
## $ atv             <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "No", "Unkn~
## $ motorcycle      <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "No", "Unkn~
## $ pack_and_saddle <chr> NA, NA, "yes", "yes", "yes", "yes", "yes", ~
## $ motorized_watercraft <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "No", "No", ~
## $ non_motorized_watercraft <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "No", "No", ~
## $ cross_country_ski <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "Unknown", ~
## $ snowmobile      <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "No", "Unkn~
## $ snowshoe        <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "Unknown", ~
## $ dog_sled        <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "Unknown", ~
## $ pets_allowed    <chr> "Yes", NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ county          <chr> "Stevens/Pend Oreille", "Whitman", "Whitman~
## $ municipality    <chr> NA, "WADOT", "County", "WADOT", "WADOT", NA~
## $ us_congressional_district <dbl> 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5~
## $ state_legislative_district <dbl> 7, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9~
## $ management_agency <chr> "WA Department of Natural Resources", "WA D~
## $ year_constructed <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ last_maintenance <lg1> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ right_of_way    <lg1> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ comments        <chr> NA, "Built for new development. Still in pr~
## $ informational_url <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "http://whi~
## $ global_id       <chr> "f1bf53f8-361e-4205-8226-b0b9d202d119", "9c~
## $ shape_length    <dbl> 33.25289, 2956.49668, 22314.13349, 3280.458~
## $ trail_status    <chr> "Existing", "Existing", "Existing", "Existi~
```

```
skim(trails)
```

Table 1: Data summary

Name	trails
Number of rows	22338
Number of columns	51
Column type frequency:	
character	40
logical	2
numeric	9

Group variables

None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
trail_name	156	0.99	1	74	0	6204	0
trail_alternate_name	18904	0.15	2	65	0	460	0
segment_id	21682	0.03	1	48	0	614	0
trail_system_type	1583	0.93	5	9	0	9	0
trail_system_name	8421	0.62	3	52	0	566	0
national_trail_designation	22093	0.01	3	31	0	7	0
trail_surface	2887	0.87	3	27	0	30	0
trail_condition	19044	0.15	1	29	0	14	0
trail_type	3191	0.86	4	20	0	13	0
trail_class	16847	0.25	3	20	0	11	0
primary_use	4159	0.81	4	20	0	21	0
trail_usage	21599	0.03	3	9	0	4	0
seasonality	18595	0.17	6	13	0	5	0
season_notes	21834	0.02	9	25	0	6	0
directionality	21834	0.02	2	20	0	15	0
accessibility_status	14666	0.34	7	14	0	4	0
special_management_area	16764	0.25	3	40	0	23	0
interpretive_trail	21512	0.04	2	7	0	4	0
rail_trail	21361	0.04	2	3	0	4	0
hiking_walking	2065	0.91	2	7	0	5	0
bicycle	8813	0.61	2	7	0	5	0
e_bike	19530	0.13	2	7	0	4	0
x4wd	13103	0.41	2	7	0	4	0
atv	10142	0.55	2	7	0	4	0
motorcycle	10682	0.52	2	11	0	6	0
pack_and_saddle	8634	0.61	2	11	0	9	0
motorized_watercraft	16192	0.28	2	7	0	4	0
non_motorized_watercraft	13981	0.37	2	7	0	4	0
cross_country_ski	14329	0.36	2	7	0	4	0
snowmobile	14331	0.36	2	7	0	4	0
snowshoe	15508	0.31	2	7	0	5	0
dog_sled	14844	0.34	2	7	0	4	0
pets_allowed	19154	0.14	2	7	0	5	0
county	195	0.99	4	30	0	91	0
municipality	19823	0.11	3	21	0	18	0
management_agency	224	0.99	3	49	0	139	0
comments	14718	0.34	1	266	0	427	0
informational_url	18180	0.19	3	120	0	333	0
global_id	0	1.00	36	36	0	22338	0
trail_status	0	1.00	4	12	0	8	0

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
last_maintenance	22338	0	NaN	:

skim_variable	n_missing	complete_rate	mean	count
right_of_way	22338	0	NaN	:

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
objectid	0	1.00	49684.266793.64	32305	44255.2549849.5055445.7561983.0					
trail_number	20267	0.09	928.24	1264.02	0	121.50	533.00	1291.50	12110.0	
segment_length_miles	257	0.99	0.99	4.41	-1	0.05	0.17	0.65	381.9	
trail_width	19493	0.13	7.87	4.77	0	4.00	8.00	10.00	35.0	
speed_limit_mph	22274	0.00	14.69	1.22	10	15.00	15.00	15.00	15.0	
us_congressional_district	234	0.99	5.69	21.06	1	2.00	5.00	8.00	910.0	
state_legislative_district	227	0.99	1828.18	220409.61	1	10.00	19.00	39.00	32364346.0	
year_constructed	22297	0.00	2015.66	19.03	1900	2015.00	2020.00	2022.00	2023.0	
shape_length	1	1.00	1582.14	7017.69	0	86.08	280.38	1041.95	614606.5	

```
# 2. SHARED BASE CLEANING
#
# Rules applied here affect every sub-dataset equally:
# a) Trim whitespace in all character columns
# b) Normalize placeholder strings to NA
# c) Keep only active / existing trail segments

trails_base <- trails |>

# whitespace
mutate(across(where(is.character), str_squish)) |>

# Blank / placeholder -> NA
mutate(across(where(is.character), ~ na_if(., ""))) |>
mutate(across(where(is.character), ~ na_if(., "N/A"))) |>
mutate(across(where(is.character), ~ na_if(., "n/a"))) |>
mutate(across(where(is.character), ~ na_if(., "None"))) |>
mutate(across(where(is.character), ~ na_if(., "Unknown"))) |>
mutate(across(where(is.character), ~ na_if(., "Not Specified"))) |>

# Active trails only (remove proposed / decommissioned / closed)
filter(is.na(trail_status) |
       str_to_lower(trail_status) %in% c("existing", "open"))

cat("Base after global clean:", nrow(trails_base), "rows\n")
```

Base after global clean: 22303 rows

```
# =====
# SUB-DATASET 1 - COMPARATIVE
# Question: How do trail surface types compare across primary use categories
#           in terms of average segment length and trail width?
#
# Variables:
```

```

# objectid          - unique row ID
# trail_name        - for reference / labeling
# surface_cat       - derived: broad surface group (Paved, Gravel, etc.)
# use_cat           - derived: broad primary use group (Hiking, Biking, etc.)
# segment_length_miles - continuous outcome variable (length comparison)
# trail_width        - continuous outcome variable (width comparison)
# =====

df_q1 <- trails_base |>

# 1. Keep rows with all four required raw variables
filter(
  !is.na(trail_surface),
  !is.na(primary_use),
  !is.na(segment_length_miles),
  !is.na(trail_width)
) |>

# 2. Remove physically impossible / extreme lengths
#   negative length (-1.0 min in raw) = data entry error
#   > 50 miles = almost certainly a cumulative route total, not one segment
filter(segment_length_miles > 0, segment_length_miles <= 50) |>

# 3. Remove zero-width trails (unrecorded, not a real trail width)
filter(trail_width > 0) |>

# 4. Collapse trail_surface into surface_cat (6 broad groups)
mutate(surface_cat = case_when(
  str_detect(str_to_lower(trail_surface), "paved|asphalt|concrete") ~ "Paved",
  str_detect(str_to_lower(trail_surface), "gravel|crushed") ~ "Gravel/Crushed",
  str_detect(str_to_lower(trail_surface), "native|dirt|natural|earth") ~ "Native/Dirt",
  str_detect(str_to_lower(trail_surface), "wood|boardwalk") ~ "Wood/Boardwalk",
  str_detect(str_to_lower(trail_surface), "snow|ice") ~ "Snow/Ice",
  TRUE ~ "Other"
)) |>

# 5. Collapse primary_use into use_cat (6 broad groups)
mutate(use_cat = case_when(
  str_detect(str_to_lower(primary_use), "hik|walk|foot") ~ "Hiking/Walking",
  str_detect(str_to_lower(primary_use), "bik|cycl|mountain bike") ~ "Biking",
  str_detect(str_to_lower(primary_use), "atv|ohv|motoriz|4wd|jeep") ~ "Motorized",
  str_detect(str_to_lower(primary_use), "horse|equestrian|pack") ~ "Equestrian",
  str_detect(str_to_lower(primary_use), "ski|snow|snowshoe") ~ "Winter Use",
  str_detect(str_to_lower(primary_use), "water|canoe|kayak") ~ "Water",
  TRUE ~ "Other/Multi"
)) |>

# 6. Keep only the 6 variables needed for Q1
select(
  objectid,
  trail_name,
  surface_cat,          # grouping variable (x-axis / facet)
  use_cat,              # grouping variable (color / facet)

```

```

    segment_length_miles, # outcome 1
    trail_width          # outcome 2
  )

cat("\n-- df_q1 (Comparative) --\n")

```

```

##
## -- df_q1 (Comparative) --

```

```
cat("Rows:", nrow(df_q1), "| Cols:", ncol(df_q1), "\n")
```

```
## Rows: 2087 | Cols: 6
```

```
cat("Columns:", paste(names(df_q1), collapse = ", "), "\n")
```

```
## Columns: objectid, trail_name, surface_cat, use_cat, segment_length_miles, trail_width
```

```
tabyl(df_q1, surface_cat) |> print()
```

```

##      surface_cat      n    percent
## Gravel/Crushed    59 0.02827024
##   Native/Dirt    639 0.30618112
##           Other  1032 0.49448970
##           Paved   297 0.14230954
##   Wood/Boardwalk   60 0.02874940

```

```
tabyl(df_q1, use_cat) |> print()
```

```

##           use_cat      n    percent
##           Biking       7 0.0033540968
## Hiking/Walking  2035 0.9750838524
##           Motorized     1 0.0004791567
##           Other/Multi   44 0.0210828941

```

```

# =====
# SUB-DATASET 2 - CORRELATION
# Question: Does trail width correlate with the number of permitted use types?
#
# Variables:
#   objectid      - unique row ID
#   trail_name     - for reference
#   trail_width    - continuous predictor (x-axis)
#   num_uses       - derived: count of permitted use types (y-axis)
#   motor_type     - derived: Motorized Only / Non-Motorized Only / Mixed
#   [13 use flags] - binary 0/1 columns kept for per-use breakdown charts
# =====

# All 13 use-type columns present in the raw data (raw values: "Yes" / "No" / NA)
use_cols <- c(

```

```

"hiking_walking", "bicycle", "e_bike",
"x4wd", "atv", "motorcycle",
"pack_and_saddle",
"motorized_watercraft", "non_motorized_watercraft",
"cross_country_ski", "snowmobile", "snowshoe", "dog_sled"
)

df_q2 <- trails_base |>

# 1. Need a valid, non-zero trail width
filter(!is.na(trail_width), trail_width > 0) |>

# 2. Binarize all use-type columns: "Yes" = 1, everything else = 0
#    NA is treated as 0 (not recorded = not permitted for this analysis)
mutate(across(
  all_of(use_cols),
  ~ if_else(str_to_lower(.) == "yes", 1L, 0L, missing = 0L)
)) |>

# 3. Derive num_uses = total number of permitted use types per segment
mutate(num_uses = rowSums(across(all_of(use_cols)), na.rm = TRUE)) |>

# 4. Drop segments with zero uses recorded (nothing to correlate against)
filter(num_uses > 0) |>

# 5. Classify each segment as Motorized / Non-Motorized / Mixed
mutate(
  has_motorized = (x4wd + atv + motorcycle + snowmobile) > 0,
  has_nonmotorized = (hiking_walking + bicycle + cross_country_ski +
    snowshoe + pack_and_saddle) > 0,
  motor_type = case_when(
    has_motorized & !has_nonmotorized ~ "Motorized Only",
    !has_motorized & has_nonmotorized ~ "Non-Motorized Only",
    has_motorized & has_nonmotorized ~ "Mixed",
    TRUE ~ "Unclassified"
  )
) |>

# 6. Keep only the variables needed for Q2
select(
  objectid,
  trail_name,
  trail_width,          # continuous predictor
  num_uses,             # derived count - response variable
  motor_type,           # color / facet grouping
  all_of(use_cols)      # individual flags for breakdown analysis
)

cat("\n-- df_q2 (Correlation) --\n")

```

```

##
## -- df_q2 (Correlation) --

```



```
cat("Rows:", nrow(df_q2), "| Cols:", ncol(df_q2), "\n")
```

```
## Rows: 2516 | Cols: 18
```

```
cat("Columns:", paste(names(df_q2), collapse = ", "), "\n")
```

```
## Columns: objectid, trail_name, trail_width, num_uses, motor_type, hiking_walking, bicycle, e_bike, x
```

```
summary(df_q2$trail_width) |> print()
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1.000   5.000   8.000   8.353  10.000  35.000
```

```
summary(df_q2$num_uses) |> print()
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1.00   1.00   1.00   1.38   2.00   3.00
```

```
tabyl(df_q2, motor_type) |> print()
```

```
##           motor_type    n    percent
##      Motorized Only     1 0.0003974563
##      Non-Motorized Only 2515 0.9996025437
```

```
# =====
# SUB-DATASET 3 - GEOGRAPHIC
# Question: Which WA counties have the highest density of motorized vs.
#           non-motorized trails, and are motorized trails concentrated
#           in Eastern WA?
#
# Variables:
#   objectid      - unique row ID
#   trail_name    - for reference
#   county        - cleaned county name (geographic grouping unit)
#   region        - derived: "Eastern WA" or "Western WA"
#   is_motorized  - derived: "Motorized" or "Non-Motorized"
#   segment_length_miles - for aggregating total trail miles per county
# =====

# 20 counties east of the Cascades
eastern_wa_counties <- c(
  "Adams", "Asotin", "Benton", "Chelan", "Columbia", "Douglas",
  "Ferry", "Franklin", "Garfield", "Grant", "Kittitas", "Klickitat",
  "Lincoln", "Okanogan", "Pend Oreille", "Spokane", "Stevens",
  "Walla Walla", "Whitman", "Yakima"
)

df_q3 <- trails_base |>
```

```

# 1. Need county and a valid segment length
filter(!is.na(county), !is.na(segment_length_miles)) |>
filter(segment_length_miles > 0, segment_length_miles <= 50) |>

# 2. Standardize county names
# Remove trailing "County" suffix, title-case, trim spaces
# e.g. "king county" -> "King", "PIERCE COUNTY" -> "Pierce"
mutate(county = str_remove(county, regex("\\s*county$", ignore_case = TRUE)) |>
  str_to_title() |>
  str_squish()) |>

# 3. Binarize use columns to classify motorized vs. non-motorized
mutate(across(
  all_of(use_cols),
  ~ if_else(str_to_lower(.) == "yes", 1L, 0L, missing = 0L)
)) |>
mutate(
  is_motorized = if_else(
    (atv + motorcycle + x4wd + snowmobile) > 0,
    "Motorized",
    "Non-Motorized"
  )
) |>

# 4. Classify Eastern vs. Western WA based on county
mutate(region = if_else(county %in% eastern_wa_counties,
  "Eastern WA", "Western WA")) |>

# 5. Keep only the 6 variables needed for Q3
select(
  objectid,
  trail_name,
  county,           # geographic grouping unit
  region,           # Eastern WA / Western WA
  is_motorized,     # Motorized / Non-Motorized
  segment_length_miles # for total-miles-per-county aggregation
)

cat("\n-- df_q3 (Geographic) --\n")

```

```

##
## -- df_q3 (Geographic) --

```

```

cat("Rows:", nrow(df_q3), "| Cols:", ncol(df_q3), "\n")

```

```

## Rows: 22003 | Cols: 6

```

```

cat("Columns:", paste(names(df_q3), collapse = ", "), "\n")

```

```

## Columns: objectid, trail_name, county, region, is_motorized, segment_length_miles

```

```
cat("Counties represented:", n_distinct(df_q3$county), "\n")
```

```
## Counties represented: 90
```

```
tabyl(df_q3, region, is_motorized) |> print()
```

```
##      region Motorized Non-Motorized
## Eastern WA      800      4068
## Western WA     1193     15942
```

```
# =====
# SUB-DATASET 4 - TIME / TREND
# Question: How has trail construction volume changed over time (by decade),
#           and which trail types or agencies drove recent growth?
#
# Variables:
#   objectid           - unique row ID
#   trail_name         - for reference
#   year_constructed   - raw year (continuous, for annual trend line)
#   decade             - derived: floored to decade (1990, 2000, 2010 ...)
#   era                - derived: labeled period factor (Pre-1970, 1990s, etc.)
#   trail_type_cat     - derived: broad trail type group
#   agency_clean       - derived: top agencies kept; rare ones -> "Other Agency"
#   segment_length_miles - for total miles constructed per period
# =====

df_q4 <- trails_base |>

# 1. Keep only rows with a recorded construction year
# Note: ~22,297 of 22,338 rows are NA here - this sub-dataset will be small
filter(!is.na(year_constructed)) |>

# 2. Remove implausible years (dataset spans 1900-2023 in raw summary)
filter(year_constructed >= 1900, year_constructed <= 2025) |>

# 3. Remove invalid segment lengths
filter(!is.na(segment_length_miles),
       segment_length_miles > 0,
       segment_length_miles <= 50) |>

# 4. Derive decade and era period labels
mutate(
  decade = (year_constructed %/% 10) * 10,
  era     = case_when(
    year_constructed < 1970 ~ "Pre-1970",
    year_constructed < 1990 ~ "1970s-80s",
    year_constructed < 2000 ~ "1990s",
    year_constructed < 2010 ~ "2000s",
    year_constructed < 2020 ~ "2010s",
    TRUE                    ~ "2020s"
  ) |> factor(levels = c("Pre-1970", "1970s-80s", "1990s",
                        "2000s", "2010s", "2020s"))
```

```

) |>

# 5. Collapse trail_type into trail_type_cat (7 broad groups)
mutate(trail_type_cat = case_when(
  str_detect(str_to_lower(trail_type), "hik|foot|pedestrian|walk") ~ "Hiking/Pedestrian",
  str_detect(str_to_lower(trail_type), "multi|shared|paved") ~ "Multi-Use Paved",
  str_detect(str_to_lower(trail_type), "mtn|mountain bike|bike") ~ "Mountain Bike",
  str_detect(str_to_lower(trail_type), "motoriz|ohv|atv|ohm") ~ "Motorized OHV",
  str_detect(str_to_lower(trail_type), "equestrian|horse|pack") ~ "Equestrian",
  str_detect(str_to_lower(trail_type), "winter|snow|ski") ~ "Winter",
  str_detect(str_to_lower(trail_type), "water") ~ "Water",
  TRUE ~ "Other/Unknown"
)) |>

# 6. Collapse rare management agencies into "Other Agency"
#   Threshold: keep agencies with >= 30 segments in this filtered dataset
#   This preserves the top ~5-8 agencies for a readable trend chart
add_count(management_agency, name = "n_agency") |>
mutate(agency_clean = if_else(n_agency >= 30,
                             management_agency,
                             "Other Agency")) |>

select(-n_agency) |>

# 7. Keep only the 8 variables needed for Q4
select(
  objectid,
  trail_name,
  year_constructed,      # raw year: for continuous trend line
  decade,               # grouped: bar / area chart by decade
  era,                  # ordered factor: readable axis labels
  trail_type_cat,       # color / facet: what type was being built?
  agency_clean,         # color / facet: who was building?
  segment_length_miles  # for total miles constructed per period
)

cat("\n-- df_q4 (Time/Trend) --\n")

##
## -- df_q4 (Time/Trend) --

cat("Rows:", nrow(df_q4), "| Cols:", ncol(df_q4), "\n")

## Rows: 40 | Cols: 8

cat("Columns:", paste(names(df_q4), collapse = ", "), "\n")

## Columns: objectid, trail_name, year_constructed, decade, era, trail_type_cat, agency_clean, segment_

tabyl(df_q4, era)      |> print()

##           era  n percent

```

```
##   Pre-1970  1  0.025
##  1970s-80s  0  0.000
##    1990s  0  0.000
##    2000s  1  0.025
##    2010s 15  0.375
##    2020s 23  0.575
```

```
tabyl(df_q4, trail_type_cat) |> print()
```

```
##   trail_type_cat  n percent
##   Other/Unknown 40         1
```

```
tabyl(df_q4, agency_clean) |> print()
```

```
##           agency_clean  n percent
## City of Bonney Lake 30    0.75
##      Other Agency 10    0.25
```

```
view(df_q1)
view(df_q2)
view(df_q3)
view(df_q4)
```

```
# 4. FINAL SUMMARY
```

```
cat("\n=== CLEANING SUMMARY ===\n")
```

```
##
## === CLEANING SUMMARY ===
```

```
cat("Raw rows          :", nrow(trails_raw), "\n")
```

```
## Raw rows          : 22338
```

```
cat("After base cleaning :", nrow(trails_base), "\n\n")
```

```
## After base cleaning : 22303
```

```
cat("df_q1 Comparative  :", nrow(df_q1), "rows x", ncol(df_q1), "vars\n")
```

```
## df_q1 Comparative  : 2087 rows x 6 vars
```

```
cat("df_q2 Correlation  :", nrow(df_q2), "rows x", ncol(df_q2), "vars\n")
```

```
## df_q2 Correlation  : 2516 rows x 18 vars
```

```
cat("df_q3 Geographic      :", nrow(df_q3), "rows x", ncol(df_q3), "vars\n")
```

```
## df_q3 Geographic      : 22003 rows x 6 vars
```

```
cat("df_q4 Time/Trend      :", nrow(df_q4), "rows x", ncol(df_q4), "vars\n")
```

```
## df_q4 Time/Trend      : 40 rows x 8 vars
```

```
cat("
Key cleaning decisions:
  * segment_length_miles <= 0 or > 50 dropped (errors / cumulative totals)
  * trail_width = 0 dropped (physically impossible)
  * year_constructed outside 1900-2025 dropped (implausible)
  * Blank / 'Unknown' / 'N/A' strings normalized to NA (all char cols)
  * trail_status filtered to active / existing trails only
  * Use-type Yes/No columns binarized (NA treated as 0 = not permitted)
  * County names: 'County' suffix stripped, title-cased, whitespace trimmed
  * Surface / use / trail_type collapsed to broad readable categories
  * Agencies with < 30 segments in Q4 collapsed to 'Other Agency'
")
```

```
##
## Key cleaning decisions:
##   * segment_length_miles <= 0 or > 50 dropped (errors / cumulative totals)
##   * trail_width = 0 dropped (physically impossible)
##   * year_constructed outside 1900-2025 dropped (implausible)
##   * Blank / 'Unknown' / 'N/A' strings normalized to NA (all char cols)
##   * trail_status filtered to active / existing trails only
##   * Use-type Yes/No columns binarized (NA treated as 0 = not permitted)
##   * County names: 'County' suffix stripped, title-cased, whitespace trimmed
##   * Surface / use / trail_type collapsed to broad readable categories
##   * Agencies with < 30 segments in Q4 collapsed to 'Other Agency'
```

EDA Visualizations

```
# library
library(scales)      # label_comma(), percent_format()
```

```
##
## Attaching package: 'scales'

## The following object is masked from 'package:purrr':
##
##   discard
```

```
## The following object is masked from 'package:readr':
##
##   col_factor
```

```
library(ggtext)      # element_markdown() for styled axis/title text

# Shared minimal theme used across all four plots
theme_trails <- function() {
  theme_minimal(base_size = 12) +
    theme(
      plot.title       = element_text(face = "bold", size = 14, hjust = 0),
      plot.subtitle    = element_text(size = 10, colour = "grey40", hjust = 0,
                                       margin = margin(b = 10)),
      plot.caption     = element_text(size = 8, colour = "grey55", hjust = 0),
      axis.title       = element_text(size = 10),
      legend.position  = "bottom",
      legend.title     = element_text(size = 9),
      panel.grid.minor = element_blank()
    )
}
```

```
# =====
# Q1 - COMPARATIVE
# "How do trail surface types compare across primary use categories in terms
# of average segment length and trail width?"
#
# Chart choice: Grouped box plot
# x      = surface_cat (the comparison groups)
# y      = segment_length_miles (continuous outcome)
# color = surface_cat (for visual distinction)
# note  : use_cat is nearly all Hiking/Walking (97.5%) so we annotate
#         rather than facet, to avoid near-empty panels
# =====

# Summarise median length per surface for annotation
q1_medians <- df_q1 |>
  group_by(surface_cat) |>
  summarise(
    med_length = median(segment_length_miles),
    med_width  = median(trail_width),
    n          = n(),
    .groups    = "drop"
  ) |>
  arrange(desc(med_length))

# Plot
plot_q1 <- df_q1 |>
  # Re-order surface_cat by median length (most informative ordering)
  mutate(surface_cat = fct_reorder(surface_cat,
                                    segment_length_miles,
                                    .fun = median)) |>

  ggplot(aes(x = surface_cat,
             y = segment_length_miles,
```

```

    fill = surface_cat)) +

# Box plot: shows median, IQR, and spread
geom_boxplot(
  outlier.shape = 16,
  outlier.size = 0.8,
  outlier.alpha = 0.4,
  width = 0.55,
  colour = "grey30"
) +

# Overlay jittered points so individual segments are visible
geom_jitter(
  aes(colour = surface_cat),
  width = 0.15,
  alpha = 0.25,
  size = 0.7
) +

# Annotate median value above each box
stat_summary(
  fun = median,
  geom = "text",
  aes(label = paste0("med=", round(after_stat(y), 2), " mi")),
  vjust = -0.5,
  size = 2.8,
  colour = "grey20"
) +

# Annotate sample size below x-axis labels
geom_text(
  data = q1_medians |>
    mutate(surface_cat = fct_reorder(surface_cat, med_length)),
  aes(x = surface_cat, y = -0.5, label = paste0("n=", n)),
  size = 2.6, colour = "grey45", inherit.aes = FALSE
) +

scale_fill_brewer(palette = "Set2") +
scale_colour_brewer(palette = "Set2") +
scale_y_continuous(
  labels = label_number(suffix = " mi"),
  limits = c(-1, 20), # cap at 20 mi to show detail; outliers still shown
  oob = scales::squish
) +

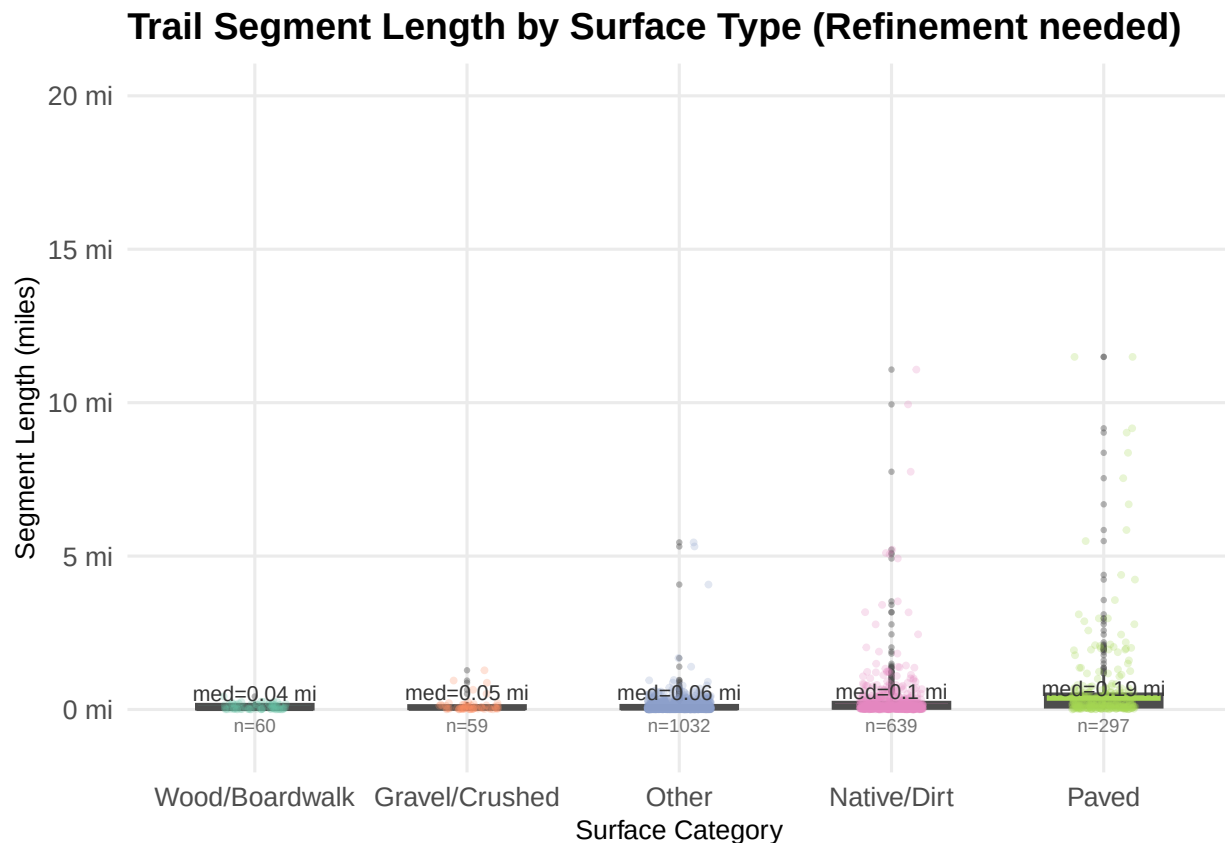
labs(
  title = "Trail Segment Length by Surface Type (Refinement needed)",
  x = "Surface Category",
  y = "Segment Length (miles)",
  fill = "Surface",
  colour = "Surface"
) +

```



```
theme_trails() +
theme(legend.position = "none") # colour already encoded on x-axis

print(plot_q1)
```



```
ggsave("plot_q1.png", plot_q1, width = 10, height = 10, dpi = 150)
```

```
# =====
# Q1 - REFINED COMPARATIVE
#
# Problems in v1:
# • All medians near 0 (0.04-0.19 mi) - boxes invisible on linear scale
# • "Other" surface (49% of data) is an uninformative catch-all
# • Jitter points dominate; box shape unreadable
#
# Fixes:
# 1. Remove "Other" surface category (uninterpretable label)
# 2. Log10 y-axis - spreads the compressed low-end distribution
# 3. Violin plot instead of box - shows the full shape at small scales
# 4. Overlay median dot + IQR bar for a clean summary layer
# 5. Add median label in readable position (not squished at zero)
# =====

q1_refined <- df_q1 |>
  filter(surface_cat != "Other") |> # remove uninformative category
```

```

filter(segment_length_miles >= 0.01)          # log10 requires > 0; drop micro-segments

# Reorder by median for meaningful left-to-right comparison
q1_refined <- q1_refined |>
  mutate(surface_cat = fct_reorder(surface_cat, segment_length_miles, median))

# Summary stats for annotation layer
q1_stats <- q1_refined |>
  group_by(surface_cat) |>
  summarise(
    med    = median(segment_length_miles),
    q25    = quantile(segment_length_miles, 0.25),
    q75    = quantile(segment_length_miles, 0.75),
    n      = n(),
    .groups = "drop"
  )

plot_q1_refined <- ggplot(q1_refined,
  aes(x = surface_cat, y = segment_length_miles,
    fill = surface_cat, colour = surface_cat)) +

# Violin: shows the full distribution shape on log scale
  geom_violin(
    alpha    = 0.35,
    linewidth = 0.4,
    trim     = TRUE,
    scale    = "width" # same visual width per group regardless of n
  ) +

# Median point
  geom_point(
    data = q1_stats,
    aes(x = surface_cat, y = med),
    size = 3, shape = 21,
    fill = "white", colour = "grey20",
    stroke = 1.2,
    inherit.aes = FALSE
  ) +

# IQR range bar
  geom_linerange(
    data = q1_stats,
    aes(x = surface_cat, ymin = q25, ymax = q75),
    linewidth = 1.2, colour = "grey20",
    inherit.aes = FALSE
  ) +

# Median value label - positioned to the right of each violin
  geom_text(
    data = q1_stats,
    aes(x = surface_cat, y = med,
      label = paste0("med=", round(med, 2), " mi\nn=", comma(n))),
    hjust = -0.15,

```

```

    vjust = 0.5,
    size = 2.9,
    colour = "grey25",
    inherit.aes = FALSE
  ) +

  # Log scale: reveals the spread that was invisible on linear scale
  scale_y_log10(
    breaks = c(0.01, 0.05, 0.1, 0.5, 1, 5, 10, 50),
    labels = label_number(suffix = " mi", accuracy = 0.01)
  ) +

  scale_fill_brewer(palette = "Set2") +
  scale_colour_brewer(palette = "Set2") +

  coord_flip() + # horizontal violins are easier to read with text labels

  labs(
    title = "Segment Length Distribution by Surface Type",
    subtitle = "Log scale reveals variation invisible on linear axis.",
    x = NULL,
    y = "Segment Length (miles, log scale)"
  ) +

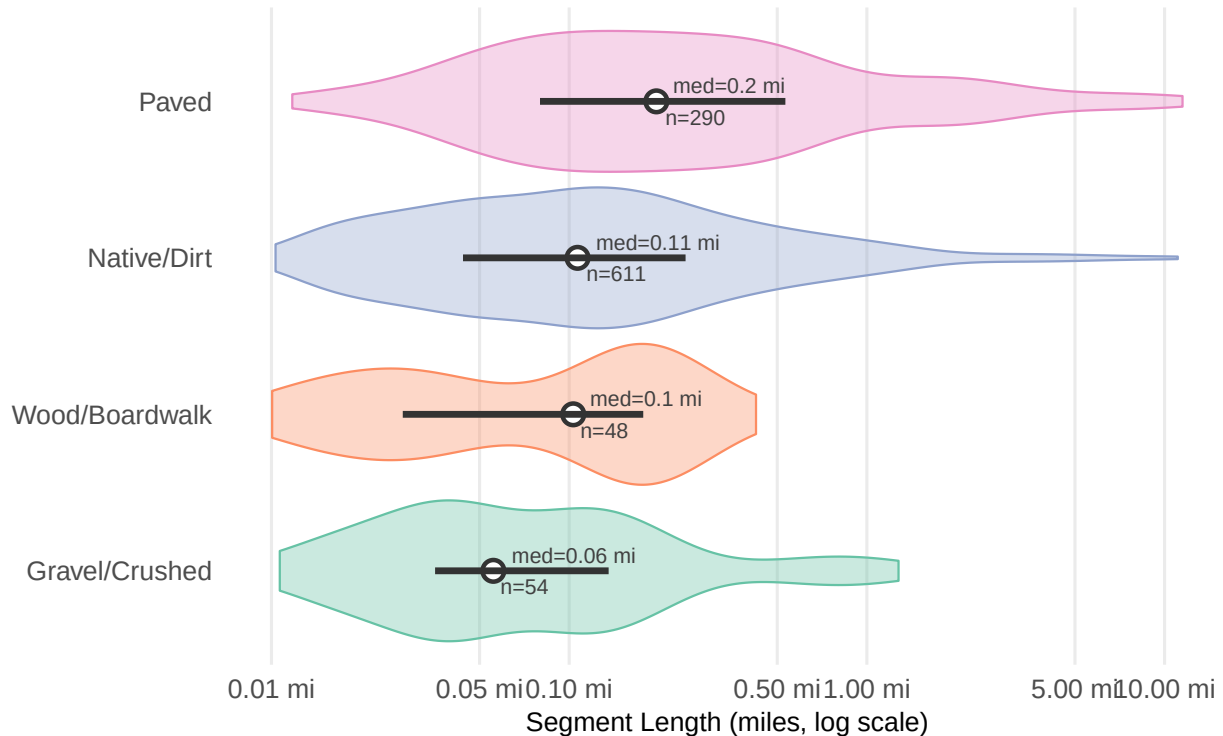
  theme_trails() +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "none",
    panel.grid.major.y = element_blank()
  )

print(plot_q1_refined)

```

Segment Length Distribution by Surface Type

Log scale reveals variation invisible on linear axis.



```
ggsave("plot_q1_refined.png", plot_q1_refined, width = 9, height = 5.5, dpi = 150)
```

```
# =====
# Q2 - CORRELATION
# "Does trail width correlate with the number of permitted use types?"
#
# Chart choice: Scatter plot + linear smoother + marginal rug
# x      = trail_width  (continuous predictor)
# y      = num_uses     (discrete count, jittered)
# color = motor_type    (Motorized Only / Non-Motorized Only / Mixed)
#
# Data note from cleaning output:
# num_uses max = 3, median = 1 - distribution is very right-skewed
# motor_type is almost entirely "Non-Motorized Only" (2515 / 2516)
# + we keep motor_type as colour but note the imbalance in subtitle
# =====

plot_q2 <- df_q2 |>
  # Jitter y slightly so overlapping points separate
  ggplot(aes(x = trail_width,
             y = num_uses,
             colour = motor_type)) +

  # Points - jitter vertically since num_uses is discrete (1, 2, 3)
  geom_jitter(
    height = 0.15,
```

```

width = 0.1,
alpha = 0.35,
size = 1.2
) +

# Linear smoother per motor_type group
geom_smooth(
  method = "lm",
  formula = y ~ x,
  se = TRUE,
  linewidth = 0.9,
  alpha = 0.15
) +

# Rug plot on x-axis to show trail width density
geom_rug(
  sides = "b",
  alpha = 0.15,
  colour = "grey40"
) +

# Pearson correlation annotation (overall)
annotate(
  "text",
  x = 28, y = 3.3,
  label = paste0("r = ",
                 round(cor(df_q2$trail_width, df_q2$num_uses,
                          use = "complete.obs"), 3)),
  size = 3.5,
  colour = "grey20",
  hjust = 1
) +

scale_colour_manual(
  values = c(
    "Non-Motorized Only" = "#2196F3",
    "Motorized Only" = "#F44336",
    "Mixed" = "#FF9800",
    "Unclassified" = "grey60"
  )
) +

scale_x_continuous(
  breaks = seq(0, 35, by = 5),
  labels = label_number(suffix = " ft")
) +

scale_y_continuous(
  breaks = 1:3,
  labels = c("1 use", "2 uses", "3 uses")
) +

labs(
  title = "Trail Width vs. Number of Permitted Uses (Refinement Needed)",

```

```

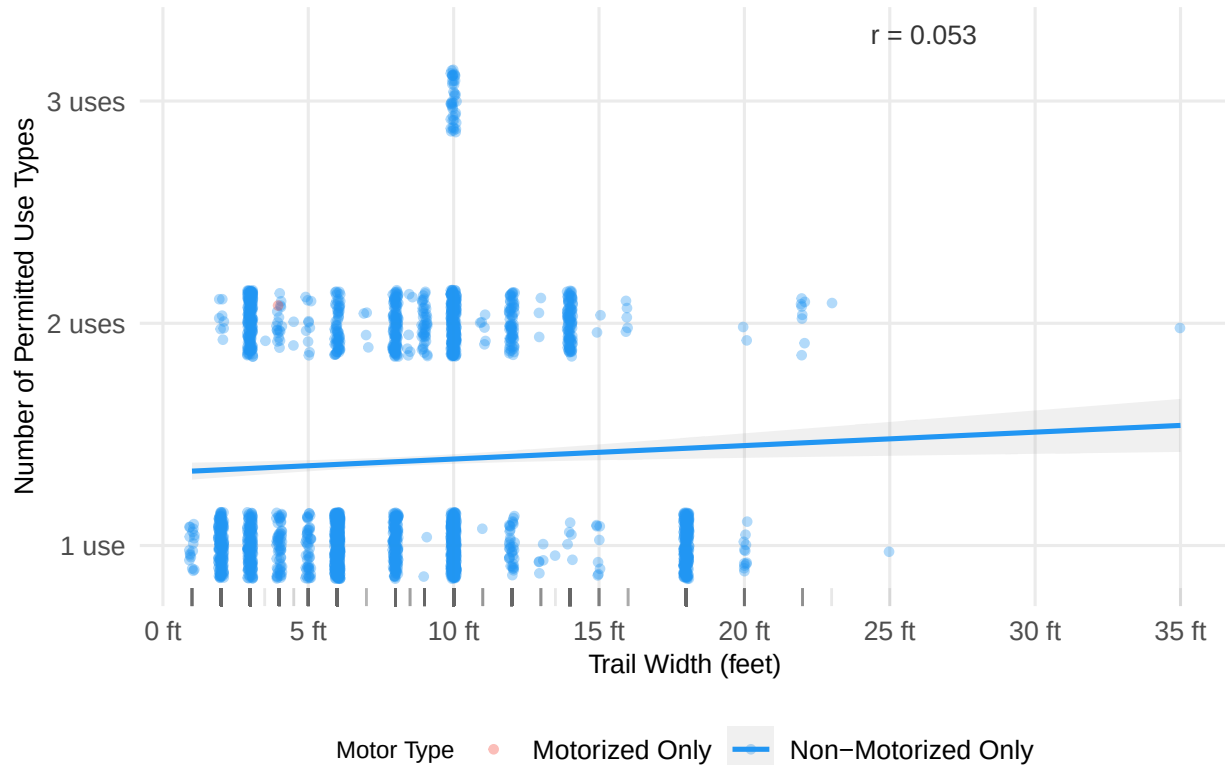
x      = "Trail Width (feet)",
y      = "Number of Permitted Use Types",
colour = "Motor Type"
) +

theme_trails()

print(plot_q2)

```

Trail Width vs. Number of Permitted Uses (Refinement Needed)



```

ggsave("plot_q2.png", plot_q2, width = 10, height = 10, dpi = 150)

```

```

# =====
# Q2 - REFINED CORRELATION
#
# Problems in v2:
# • r = 0.053 - essentially no linear correlation; scatter plot is flat
# • num_uses only has values 1, 2, 3 - scatter with 3 rows is uninformative
# • 99.96% of segments are Non-Motorized - motor_type colour is redundant
#
# Fixes:
# 1. Bin trail_width into interpretable groups (narrow / medium / wide)
# 2. Grouped bar: % of segments at each num_uses level per width bin
#    → shows whether wider trails skew toward multi-use even without linear r
# 3. Annotate actual counts on bars so readers see the n behind percentages
# 4. Facet by num_uses (1, 2, 3) so each panel is readable

```

```

# =====

q2_refined <- df_q2 |>
  mutate(
    width_bin = case_when(
      trail_width <= 4 ~ "Narrow\n(1-4 ft)",
      trail_width <= 8 ~ "Medium\n(5-8 ft)",
      trail_width <= 12 ~ "Standard\n(9-12 ft)",
      trail_width <= 20 ~ "Wide\n(13-20 ft)",
      TRUE ~ "Very Wide\n(>20 ft)"
    ) |>
    factor(levels = c("Narrow\n(1-4 ft)", "Medium\n(5-8 ft)",
                      "Standard\n(9-12 ft)", "Wide\n(13-20 ft)",
                      "Very Wide\n(>20 ft)"))
  )

# Compute % of segments at each num_uses within each width_bin
q2_pct <- q2_refined |>
  count(width_bin, num_uses) |>
  group_by(width_bin) |>
  mutate(
    pct = n / sum(n),
    bin_total = sum(n)
  ) |>
  ungroup() |>
  mutate(num_uses_label = paste0(num_uses, ifelse(num_uses == 1, " use", " uses")))

plot_q2_refined <- q2_pct |>
  ggplot(aes(x = width_bin, y = pct, fill = factor(num_uses))) +

  geom_col(position = "dodge", width = 0.7) +

  # % label on each bar
  geom_text(
    aes(label = percent(pct, accuracy = 1)),
    position = position_dodge(width = 0.7),
    vjust = -0.4,
    size = 2.7,
    colour = "grey20"
  ) +

  # Annotate total n per width bin along the x-axis
  geom_text(
    data = q2_pct |> distinct(width_bin, bin_total),
    aes(x = width_bin, y = -0.04,
        label = paste0("n=", comma(bin_total))),
    inherit.aes = FALSE,
    size = 2.6,
    colour = "grey45"
  ) +

  scale_y_continuous(
    labels = percent_format(accuracy = 1),

```

```

limits = c(-0.06, 1.05),
expand = expansion(mult = c(0, 0.05))
) +

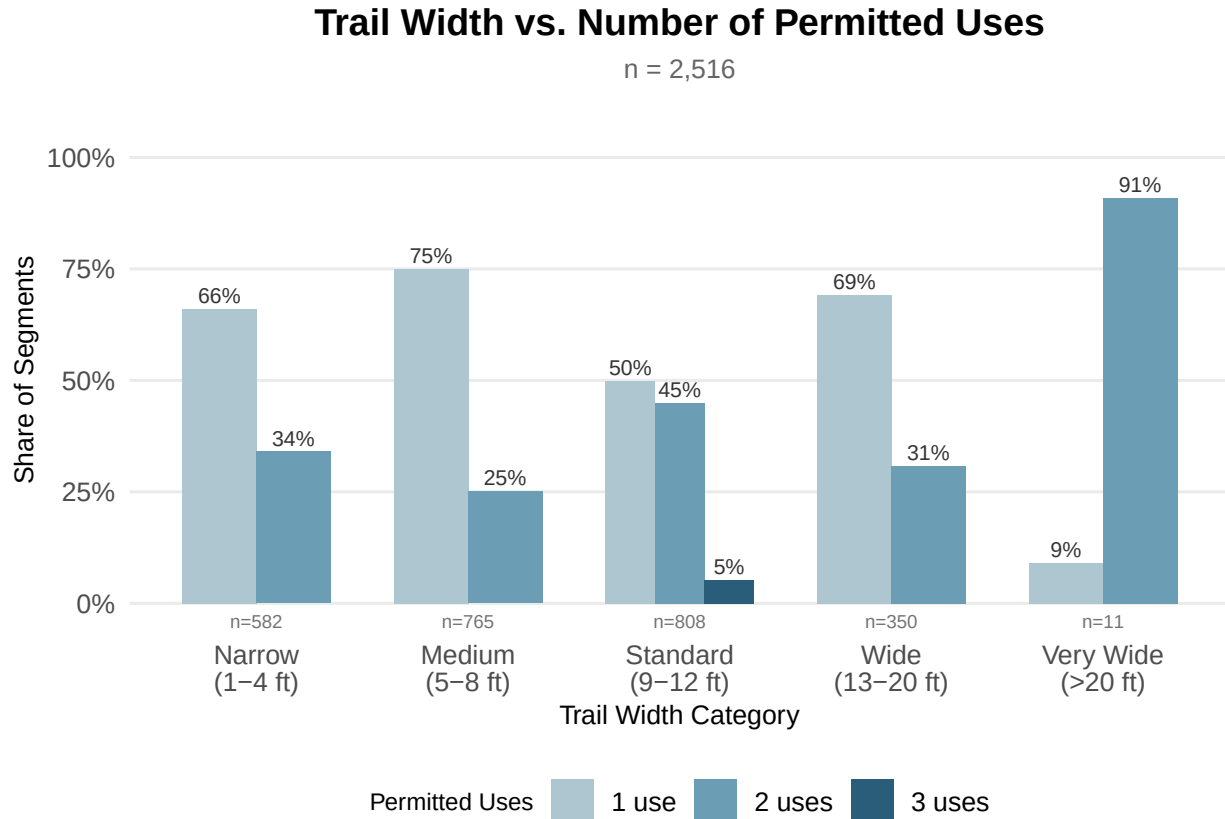
scale_fill_manual(
  values = c("1" = "#AEC6CF", "2" = "#6B9EB5", "3" = "#2A5D7A"),
  labels = c("1" = "1 use", "2" = "2 uses", "3" = "3 uses")
) +

labs(
  title = "Trail Width vs. Number of Permitted Uses",
  subtitle = "n = 2,516",
  x = "Trail Width Category",
  y = "Share of Segments",
  fill = "Permitted Uses"
) +

theme_trails() +
theme(
  plot.title = element_text(hjust = 0.5),
  plot.subtitle = element_text(hjust = 0.5),
  panel.grid.major.x = element_blank()
)

print(plot_q2_refined)

```




```
ggsave("plot_q2_refined.png", plot_q2_refined, width = 9, height = 5.5, dpi = 150)
```

```
# =====  
# Q3 - GEOGRAPHIC  
# "Which WA counties have the highest density of motorized vs. non-motorized  
# trails, and are motorized trails concentrated in Eastern WA?"  
#  
# Chart choice: Horizontal stacked bar - top 20 counties by total miles  
# x      = total trail miles (aggregated from segment_length_miles)  
# y      = county (ordered by total miles, top 20)  
# fill   = is_motorized (Motorized / Non-Motorized)  
# facet  = region (Eastern WA / Western WA) - tests the east/west hypothesis  
#  
# Data note: 90 counties; showing top 20 by total miles keeps chart readable  
# =====  
  
# Aggregate to county level  
q3_county <- df_q3 |>  
  group_by(county, region, is_motorized) |>  
  summarise(total_miles = sum(segment_length_miles, na.rm = TRUE),  
            n_segments = n(),  
            .groups = "drop") |>  
  # Compute total miles per county (for ordering)  
  group_by(county) |>  
  mutate(county_total = sum(total_miles)) |>  
  ungroup()  
  
# Keep top 20 counties by total trail miles  
top20_counties <- q3_county |>  
  distinct(county, county_total) |>  
  slice_max(county_total, n = 20) |>  
  pull(county)  
  
q3_plot_data <- q3_county |>  
  filter(county %in% top20_counties) |>  
  mutate(county = fct_reorder(county, county_total))  
  
# Proportion of motorized per region (for subtitle)  
q3_pct <- df_q3 |>  
  group_by(region, is_motorized) |>  
  summarise(n = n(), .groups = "drop") |>  
  group_by(region) |>  
  mutate(pct = n / sum(n)) |>  
  filter(is_motorized == "Motorized")  
  
# Plot  
plot_q3 <- q3_plot_data |>  
  ggplot(aes(x = total_miles,  
            y = county,  
            fill = is_motorized)) +  
  
  geom_col(position = "stack", width = 0.7) +
```

```

# Total miles label at end of each bar
geom_text(
  data = q3_plot_data |>
    group_by(county, county_total) |>
    summarise(.groups = "drop"),
  aes(x = county_total, y = county,
      label = paste0(round(county_total, 0), " mi")),
  inherit.aes = FALSE,
  hjust = -0.1, size = 2.6, colour = "grey30"
) +

# Facet by Eastern vs Western WA - directly tests the hypothesis
facet_wrap(~ region, scales = "free_y", ncol = 1) +

scale_fill_manual(
  values = c("Motorized" = "#E07B39", "Non-Motorized" = "#4A90A4"),
  labels = c("Motorized" = "Motorized", "Non-Motorized" = "Non-Motorized")
) +
scale_x_continuous(
  expand = expansion(mult = c(0, 0.18)),
  labels = label_number(suffix = " mi")
) +

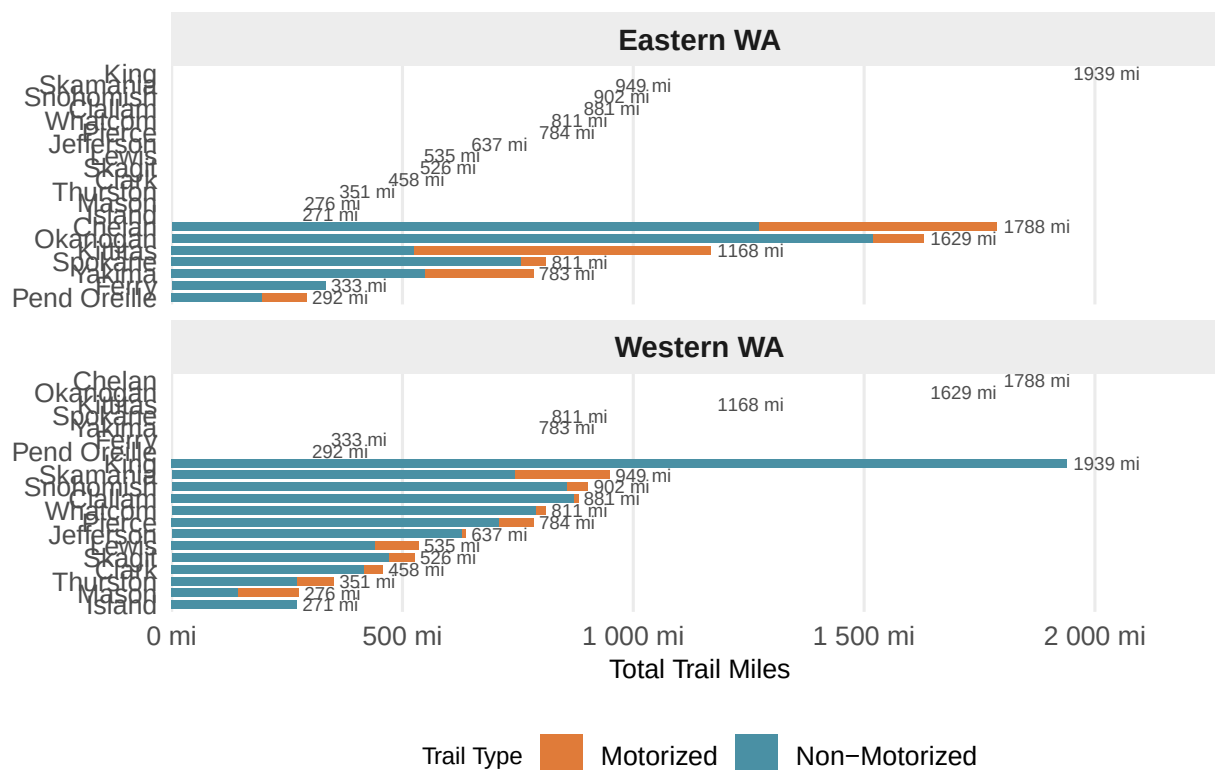
labs(
  title = "Trail Miles by County: Motorized vs. Non-Motorized(Refinement needed)",
  x = "Total Trail Miles",
  y = NULL,
  fill = "Trail Type"
) +

theme_trails() +
theme(
  strip.text = element_text(face = "bold", size = 11),
  strip.background = element_rect(fill = "grey93", colour = NA),
  panel.grid.major.y = element_blank()
)

print(plot_q3)

```

Trail Miles by County: Motorized vs. Non-Motorized(Refinem



```
ggsave("plot_q3.png", plot_q3, width = 10, height = 10, dpi = 150)
```

```
# =====
# Q3 - REFINED GEOGRAPHIC
#
# Problems in v3:
#   • Counties duplicated across both facets (Eastern counties appear in
#     Western panel and vice versa) - fct_reorder was global, not per-region
#   • Top 20 was global; some Eastern counties appeared in Western facet
#   • Labels overlapped badly
#   • x-axis labels had spaces in thousands (1 000 mi)
#
# Fixes:
#   1. Take top 10 counties WITHIN each region separately + no duplication
#   2. Use facet_grid with independent y scales so county lists don't bleed
#   3. fct_reorder within region using reorder_within() + scale_y_reordered()
#   4. Clean comma-format x-axis labels
#   5. Move mile labels inside the bar for bars > 100 mi (avoids overflow)
# =====

# Aggregate to county × region × is_motorized
q3_county <- df_q3 |>
  group_by(county, region, is_motorized) |>
  summarise(total_miles = sum(segment_length_miles, na.rm = TRUE),
    .groups = "drop") |>
  group_by(county, region) |>
```

```

mutate(county_total = sum(total_miles)) |>
ungroup()

# Top 10 counties WITHIN each region (no cross-region contamination)
top_eastern <- q3_county |>
  filter(region == "Eastern WA") |>
  distinct(county, county_total) |>
  slice_max(county_total, n = 10) |>
  pull(county)

top_western <- q3_county |>
  filter(region == "Western WA") |>
  distinct(county, county_total) |>
  slice_max(county_total, n = 10) |>
  pull(county)

q3_plot_data <- q3_county |>
  filter(
    (region == "Eastern WA" & county %in% top_eastern) |
    (region == "Western WA" & county %in% top_western)
  )

# Reorder counties within each region independently - base R approach
# (avoids needing tidytext::reorder_within)
# Strategy: build a factor level order by sorting each region separately,
# then concatenate Eastern levels first, Western levels second.
# facet_wrap(scales = "free_y") will only show the relevant subset per panel.

eastern_order <- q3_plot_data |>
  filter(region == "Eastern WA") |>
  distinct(county, county_total) |>
  arrange(county_total) |> # ascending so highest is at top of flipped axis
  pull(county)

western_order <- q3_plot_data |>
  filter(region == "Western WA") |>
  distinct(county, county_total) |>
  arrange(county_total) |>
  pull(county)

# Combined factor: Eastern levels first, then Western
# Each facet only renders its own subset so the ordering is correct per panel
q3_plot_data <- q3_plot_data |>
  mutate(county = factor(county, levels = c(eastern_order, western_order)))

# Region-level motorized % for subtitle
q3_pct <- df_q3 |>
  count(region, is_motorized) |>
  group_by(region) |>
  mutate(pct = n / sum(n)) |>
  filter(is_motorized == "Motorized") |>
  select(region, pct)

```

```

east_pct <- q3_pct |> filter(region == "Eastern WA") |> pull(pct)
west_pct <- q3_pct |> filter(region == "Western WA") |> pull(pct)

# Plot
plot_q3_refined <- q3_plot_data |>
  ggplot(aes(x = total_miles, y = county, fill = is_motorized)) +

  geom_col(position = "stack", width = 0.65) +

  # Label: inside bar if wide enough, outside if narrow
  geom_text(
    data = q3_plot_data |>
      group_by(county, region, county_total) |>
      summarise(.groups = "drop"),
    aes(x = county_total, y = county,
        label = paste0(comma(round(county_total, 0)), " mi"),
        hjust = if_else(county_total > 300, -0.08, -0.08)),
    inherit.aes = FALSE,
    size = 2.7, colour = "grey25"
  ) +

  # Independent y scale per facet so each region shows only its own counties
  facet_wrap(~ region, scales = "free_y", ncol = 1) +

  scale_fill_manual(
    values = c("Motorized" = "#E07B39", "Non-Motorized" = "#4A90A4")
  ) +

  scale_x_continuous(
    labels = label_comma(suffix = " mi"),
    expand = expansion(mult = c(0, 0.18))
  ) +

  labs(
    title = "Trail Miles by County: Motorized vs. Non-Motorized",
    subtitle = paste0(
      "Top 10 counties per region shown"
    ),
    x = "Total Trail Miles",
    y = NULL,
    fill = "Trail Type"
  ) +

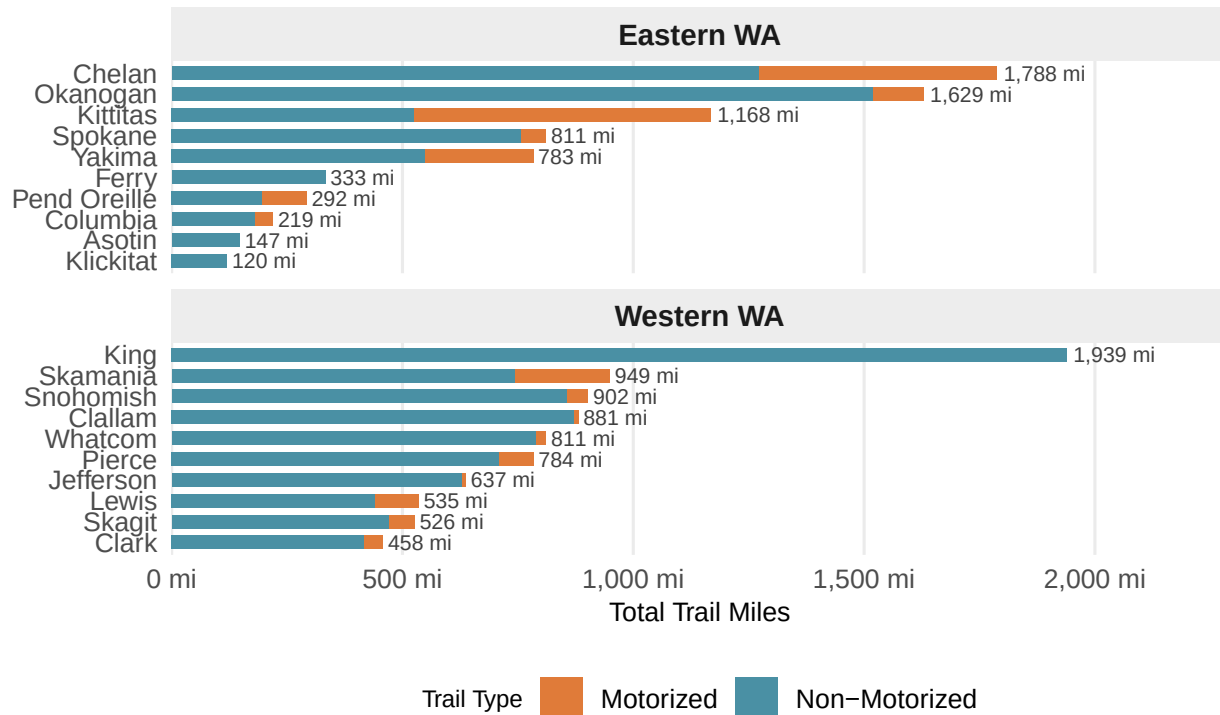
  theme_trails() +
  theme(
    plot.title = element_text(hjust = 0.5),
    strip.text = element_text(face = "bold", size = 11),
    strip.background = element_rect(fill = "grey93", colour = NA),
    panel.grid.major.y = element_blank()
  )

print(plot_q3_refined)

```

Trail Miles by County: Motorized vs. Non-Motorized

Top 10 counties per region shown



```
ggsave("plot_q3_refined.png", plot_q3_refined, width = 9, height = 9, dpi = 150)
```

```
# =====
# Q4 - TIME / TREND
# "How has trail construction volume changed over time by decade, and which
# trail types or agencies drove recent growth?"
#
# IMPORTANT data note from cleaning output:
# df_q4 has only 40 rows (year_constructed is NA for 22,297 / 22,338 rows)
# trail_type_cat = 100% "Other/Unknown" - trail_type was not recorded
# agency_clean = 75% "City of Bonney Lake", 25% "Other Agency"
# + A standard stacked bar by trail_type is meaningless here
# + Instead: bar chart of segment COUNT and total MILES by era,
# coloured by agency_clean, with a clear data-limitation note
# =====

# Aggregate by era x agency
q4_summary <- df_q4 |>
  group_by(era, agency_clean) |>
  summarise(
    n_segments = n(),
    total_miles = sum(segment_length_miles, na.rm = TRUE),
    .groups = "drop"
  )

# Plot - dual panel: segment count (left) and total miles (right)
```

```

# Pivot to long for faceting both metrics in one chart
q4_long <- q4_summary |>
  pivot_longer(cols = c(n_segments, total_miles),
               names_to = "metric",
               values_to = "value") |>
  mutate(metric = recode(metric,
                        "n_segments" = "Number of Segments",
                        "total_miles" = "Total Miles Constructed"))

plot_q4 <- q4_long |>
  ggplot(aes(x = era,
             y = value,
             fill = agency_clean)) +

  geom_col(position = "stack", width = 0.6) +

  # Value label on each bar total
  geom_text(
    data = q4_long |>
      group_by(era, metric) |>
      summarise(total = sum(value), .groups = "drop"),
    aes(x = era, y = total, label = round(total, 1)),
    inherit.aes = FALSE,
    vjust = -0.4, size = 3, colour = "grey25"
  ) +

  facet_wrap(~ metric, scales = "free_y", ncol = 2) +

  scale_fill_manual(
    values = c(
      "City of Bonney Lake" = "#5C85C4",
      "Other Agency"       = "#A8C8A0"
    )
  ) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15))) +

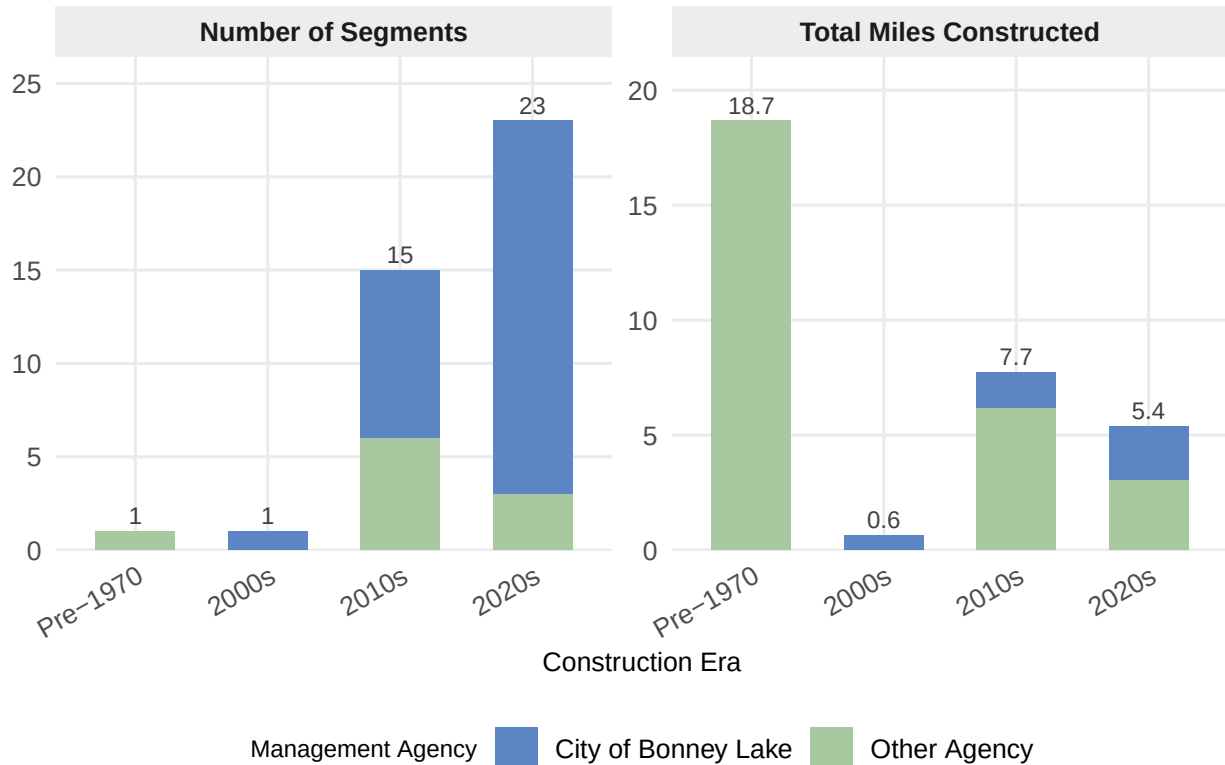
  labs(
    title = "Trail Construction Volume by Era",
    x = "Construction Era",
    y = NULL,
    fill = "Management Agency"
  ) +

  theme_trails() +
  theme(
    plot.title = element_text(hjust = 0.5),
    strip.text = element_text(face = "bold", size = 10),
    strip.background = element_rect(fill = "grey93", colour = NA),
    axis.text.x = element_text(angle = 30, hjust = 1)
  )

print(plot_q4)

```

Trail Construction Volume by Era



```
ggsave("plot_q4.png", plot_q4, width = 10, height = 10, dpi = 150)
```

Final Question: Beginner Trail Accessibility Score

```
# Question: Can we construct a county-level Beginner Trail Accessibility Score
# - weighted by non-motorized trail miles, proportion of paved or gravel
# surface, and short segment length - to rank Washington counties and identify
# the most beginner-friendly regions for first-time hikers?
#
# Score Components (equal weight, each scaled 0-1):
#   Component 1 - Non-Motorized Share:
#     proportion of trail miles that are non-motorized per county
#   Component 2 - Gentle Surface Share:
#     proportion of segments on paved or gravel/crushed surface per county
#   Component 3 - Short Segment Share:
#     proportion of segments with length <= 2 miles per county
#
```



```

# Final Score = mean(component_1, component_2, component_3), scaled 0-100

# =====

library(tidyverse)
library(janitor)
library(scales)

# 1. BUILD THE SEGMENT-LEVEL BASE
#
# We join the key variables from df_q1 (surface_cat) and df_q3 (county,
# region, is_motorized) using objectid as the key.
# df_q3 has the broadest coverage (22,003 rows) so we start there
# and left-join surface information from df_q1.
# =====

df_final <- df_q3 |>

# Join surface_cat from df_q1 on objectid
# Not all segments in df_q3 will have a surface_cat (df_q1 required
# trail_surface and trail_width to be non-missing) - those get NA
left_join(
  df_q1 |> select(objectid, surface_cat),
  by = "objectid"
) |>

# Flag 1: is the segment non-motorized?
# Already present in df_q3 as is_motorized ("Motorized" / "Non-Motorized")
mutate(
  flag_nonmotorized = if_else(is_motorized == "Non-Motorized", 1L, 0L)
) |>

# Flag 2: is the surface beginner-gentle?
# Paved and Gravel/Crushed are the most forgiving for beginners
# NA surface = unknown → treated as 0 (conservative)
mutate(
  flag_gentle_surface = case_when(
    surface_cat %in% c("Paved", "Gravel/Crushed") ~ 1L,
    is.na(surface_cat) ~ 0L,
    TRUE ~ 0L
  )
) |>

# Flag 3: is the segment short ( 2 miles)?
# 2 miles chosen as the beginner threshold:
# matches "Easy" tier in your Education Phase (1-3 miles)
# and aligns with the AllTrails dataset beginner range
mutate(
  flag_short = if_else(segment_length_miles <= 2, 1L, 0L)
)

cat("df_final rows:", nrow(df_final), "\n")

```

```
## df_final rows: 22003
```

```
cat("Columns:", paste(names(df_final), collapse = ", "), "\n")
```

```
## Columns: objectid, trail_name, county, region, is_motorized, segment_length_miles, surface_cat, flag
```

```
# 2. AGGREGATE TO COUNTY LEVEL
```

```
#
```

```
# For each county compute:
```

```
# total_miles - total trail miles (for context / weighting)
```

```
# n_segments - number of trail segments
```

```
# nonmotorized_share - proportion of segments that are non-motorized
```

```
# gentle_surface_share - proportion of segments with paved/gravel surface
```

```
# short_share - proportion of segments <= 2 miles
```

```
# =====
```

```
df_score_raw <- df_final |>
```

```
group_by(county, region) |>
```

```
summarise(
```

```
  n_segments = n(),
```

```
  total_miles = sum(segment_length_miles, na.rm = TRUE),
```

```
  nonmotorized_share = mean(flag_nonmotorized, na.rm = TRUE),
```

```
  gentle_surface_share = mean(flag_gentle_surface, na.rm = TRUE),
```

```
  short_share = mean(flag_short, na.rm = TRUE),
```

```
  .groups = "drop"
```

```
) |>
```

```
# Remove counties with fewer than 10 segments - too sparse for a reliable score
```

```
filter(n_segments >= 10)
```

```
cat("\nCounties with >= 10 segments:", nrow(df_score_raw), "\n")
```

```
##
```

```
## Counties with >= 10 segments: 51
```

```
# 3. COMPUTE BEGINNER TRAIL ACCESSIBILITY SCORE
```

```
#
```

```
# Each component is already on a 0-1 scale (it's a proportion).
```

```
# Score = mean of the three components, multiplied by 100 for readability.
```

```
#
```

```
# Weighting rationale:
```

```
# Non-motorized share - most critical for safety / beginner comfort (weight 0.4)
```

```
# Short segment share - key effort predictor for beginners (weight 0.4)
```

```
# Gentle surface share - important but data is sparse (weight 0.2)
```

```
#
```

```
# Weights sum to 1.0.
```

```
# =====
```

```
df_score <- df_score_raw |>
```

```
mutate(
```

```

# Weighted composite score (0-100)
beginner_score = (
  0.40 * nonmotorized_share +
  0.40 * short_share +
  0.20 * gentle_surface_share
) * 100
) |>

# Round components to 2 decimal places for display
mutate(across(c(nonmotorized_share, gentle_surface_share,
  short_share, beginner_score), ~ round(., 2))) |>

# Rank counties overall (1 = most beginner friendly)
mutate(rank_overall = rank(-beginner_score, ties.method = "min")) |>

# Rank within each region separately
group_by(region) |>
mutate(rank_region = rank(-beginner_score, ties.method = "min")) |>
ungroup() |>

arrange(rank_overall)

cat("\n Beginner Score Summary \n")

```

```
##
## Beginner Score Summary
```

```
summary(df_score$beginner_score)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 20.00  55.77   64.62   63.86   74.50   82.35
```

```
cat("\n Top 15 Counties Overall \n")
```

```
##
## Top 15 Counties Overall
```

```
df_score |>
  select(rank_overall, county, region, beginner_score,
    nonmotorized_share, gentle_surface_share, short_share,
    total_miles, n_segments) |>
  slice_min(rank_overall, n = 15) |>
  print(n = 15)
```

```
## # A tibble: 15 x 9
##   rank_overall county      region beginner_score nonmotorized_share
##         <int> <chr>      <chr>         <dbl>         <dbl>
## 1             1 Franklin Eastern WA         82.4             1
## 2             2 Walla Walla Eastern WA         80.0             1
## 3             3 Island    Western WA         79.5             1
## 4             4 San Juan  Western WA         78.9             1
```

```
## 5      5 King      Western WA      78.7      1
## 6      6 Clark     Western WA      78.2     0.96
## 7      7 Whatcom   Western WA      78.1      1
## 8      8 Klickitat Eastern WA      77.6      1
## 9      9 Douglas   Eastern WA      77.4      1
## 10     10 Lincoln   Eastern WA      75.8      1
## 11     11 Benton   Eastern WA      75.4      1
## 12     12 Skagit   Western WA      74.7     0.86
## 13     13 Jefferson Western WA      74.6      1
## 14     14 Grant     Eastern WA      74.4      1
## 15     15 Thurston Western WA      74.0     0.91
## # i 4 more variables: gentle_surface_share <dbl>, short_share <dbl>,
## #   total_miles <dbl>, n_segments <int>
```

4. TOP 10 PER REGION

```
# Helper: reorder county factor within each region for ggplot
# (base R version - no tidytext dependency)
```

```
eastern_order_score <- df_score |>
  filter(region == "Eastern WA") |>
  slice_min(rank_region, n = 10) |>
  arrange(beginner_score) |>
  pull(county)
```

```
western_order_score <- df_score |>
  filter(region == "Western WA") |>
  slice_min(rank_region, n = 10) |>
  arrange(beginner_score) |>
  pull(county)
```

```
df_score_top <- df_score |>
  group_by(region) |>
  slice_min(rank_region, n = 10) |>
  ungroup() |>
  mutate(county = factor(county,
                          levels = c(eastern_order_score,
                                      western_order_score)))
```

5. VISUALIZATION - BEGINNER SCORE BY COUNTY

```
#
# Chart: Horizontal bar chart, faceted by region (Eastern / Western WA)
# x     = beginner_score (0-100)
# y     = county (ordered by score within region)
# fill  = beginner_score (continuous color scale: low = muted, high = green)
# Top 10 per region shown
# =====
```

```
theme_trails <- function() {
  theme_minimal(base_size = 12) +
  theme(
    plot.title      = element_text(face = "bold", size = 14, hjust = 0.5),
    plot.subtitle   = element_text(size = 10, colour = "grey40", hjust = 0.5,
                                    margin = margin(b = 10)),
    plot.caption    = element_text(size = 8, colour = "grey55", hjust = 0),
```

```

    axis.title      = element_text(size = 10),
    legend.position = "bottom",
    panel.grid.minor = element_blank()
  )
}

plot_score <- df_score_top |>
  ggplot(aes(x = beginner_score, y = county, fill = beginner_score)) +

  geom_col(width = 0.65) +

  # Score label at end of bar
  geom_text(
    aes(label = paste0(beginner_score)),
    hjust = -0.15,
    size = 2.8,
    colour = "grey25"
  ) +

  # Component breakdown annotation inside bar
  geom_text(
    aes(label = paste0(
      "NM:", percent(nonmotorized_share, accuracy = 1),
      " Sfc:", percent(gentle_surface_share, accuracy = 1),
      " Sh:", percent(short_share, accuracy = 1)
    )),
    x = 1,
    hjust = 0,
    size = 2.3,
    colour = "white"
  ) +

  facet_wrap(~ region, scales = "free_y", ncol = 1) +

  scale_fill_gradient(
    low = "#A8C8A0", # muted green = lower score
    high = "#1B5E20", # deep green = highest score
    name = "Score",
    limits = c(0, 100)
  ) +

  scale_x_continuous(
    limits = c(0, 110),
    expand = expansion(mult = c(0, 0))
  ) +

  labs(
    title = "Beginner Trail Accessibility Score by County",
    subtitle = paste0(
      "Score (0-100) weighted: Non-Motorized(NM) 40% + Short Segment(Sh) 40% + Gentle Surface(Sfc) 20%",
      "Top 10 counties per region | Only counties with 10 trail segments included"
    ),
    x = "Beginner Trail Accessibility Score",

```

```

y          = NULL,
caption    = "Short = 2 mi | Gentle surface = Paved or Gravel/Crushed"
) +

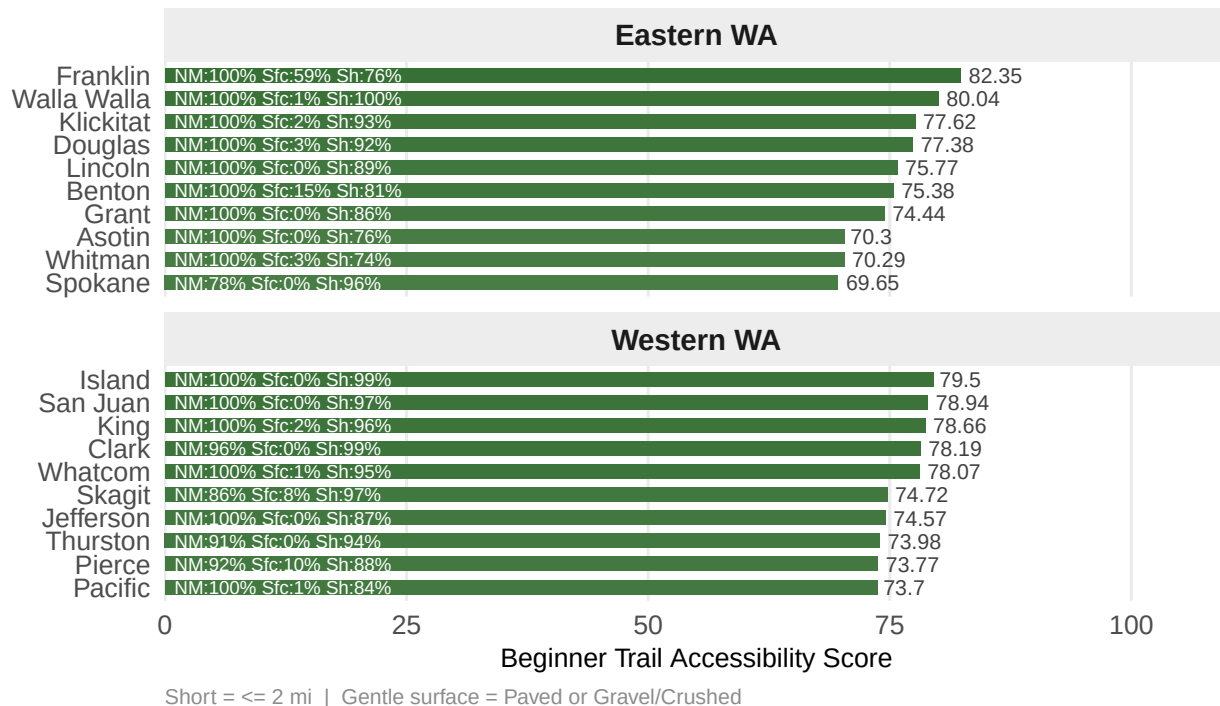
theme_trails() +
theme(
  strip.text          = element_text(face = "bold", size = 11),
  strip.background    = element_rect(fill = "grey93", colour = NA),
  panel.grid.major.y  = element_blank(),
  legend.position     = "none"
)

print(plot_score)

```

Beginner Trail Accessibility Score by County

Score (0–100) weighted: Non–Motorized(NM) 40% + Short Segment(Sh) 40% + Gentle Surface(Sfc)
 Top 10 counties per region | Only counties with >= 10 trail segments included



```

ggsave("plot_beginner_score.png", plot_score, width = 10, height = 10, dpi = 150)

```